

CIS 301 Final Exam Study Guide

The final exam for CIS 301 will be on Monday, May 11 from 2:00-3:50 pm in the usual lecture classroom (DUE 0093). It is a cumulative closed-notes/closed-computers exam and will have a similar format as the midterms, although it will be slightly longer. It will consist of a multiple choice section (similar to the in-class quizzes, Canvas practice quizzes, and multiple choice questions on previous exams) followed by open-ended problems (similar to the format of homework assignments and previous open-ended exam questions).

The final exam is worth 25% of your final grade. If it helps your grade, I will also replace your lowest score across Exam 1/Exam 2/Exam 3 with the percentage you get on the final exam.

[This repository](#) includes a selection of practice problems that are similar to some of the types of open-ended questions that might be on the final. Please note that these problems are all saved as text files (.txt extension), so Logika will not be able to verify them. I encourage you to try writing your solution to each problem on paper to practice the same environment as the final exam. Also, this list of practice problems does not cover all possible material on the final -- you will want to review all previous topics from past exams. You can find the solutions to these problems in the [lecture notes](#) from Thursday, May 7. (You can use the same link to access all lecture notes for the course.)

The final will cover:

- All topics from Exam 1:
 - Basic logical reasoning (over English statements and if-statements)
 - Logic puzzles
 - Circuits
 - Truth tables
 - Parse trees
 - Proving two propositional logic statements are equivalent (using truth tables and using natural deduction proofs)
 - Finding a counterexample to show propositional logic statements are NOT equivalent
 - DeMorgan's laws on propositional logic, including their application toward negating if-statements
 - Satisfiability - definition, how to prove a propositional logic statement is satisfiable, and how to prove one is NOT satisfiable
 - Translating between English and propositional logic
 - Arguments (premises and a proposed conclusion) in propositional logic
 - Proving an argument is invalid by finding a truth assignment that is a counterexample (one where all the premises are true but the conclusion is false)
 - Proving an argument is valid with truth tables (demonstrating that every time ALL of the premises are true, that the conclusion is also true)

- Proving an argument is valid using natural deduction proofs with the following rules:
 - And elimination (AndE1, AndE2)
 - And introduction (AndI)
 - Or introduction (OrI1, OrI2)
 - Implies elimination (ImpliesE)
- All topics on exam 2:
 - Natural deduction proofs in propositional logic, emphasizing the implies introduction rule and rules with the negation operator (not elimination, not introduction, bottom elimination, and pbc). Although they won't be specifically emphasized, you are still expected to be familiar with deduction rules with all the other operators
 - The notions of soundness and completeness
 - Sets, including: set builder notation, union, intersection, difference, subsets, sets of numbers, and general reasoning about sets
 - Predicate logic statements, including: quantifiers, interpreting predicate logic statements in English, translating English statements to predicate logic, determining whether a predicate logic statement is true or false in a particular domain, working with predicate logic statements involving sets of numbers, DeMorgan's laws, negating predicate logic statements
 - Natural deduction proofs in predicate logic using AII, AI, and Existential Introduction
 - Using natural deduction to prove two statements are equivalent
- All topics on exam 3:
 - Predicate logic proofs with the existential quantifier
 - Recursive definitions for both functions and sets
 - Proof techniques, including:
 - Mathematical induction
 - Direct (conditional) proofs
 - Contrapositive proofs
 - Proof by contradiction
 - Proofs on sets
 - Programming logic, including:
 - Assumes and asserts
 - The Subst rules and the Algebra rule
 - Verifying programs with variable assignments, variable updates, if/else statements, functions, loops, and recursion
 - Function contracts (including the meaning of preconditions and postconditions)
 - Loop invariants
 - Verifying "test code" that calls a function
- Material since exam 3:
 - Writing appropriate function contracts and loop invariants for programs that work with sequences
 - Writing function contracts for programs with global variables and global invariants

- Termination